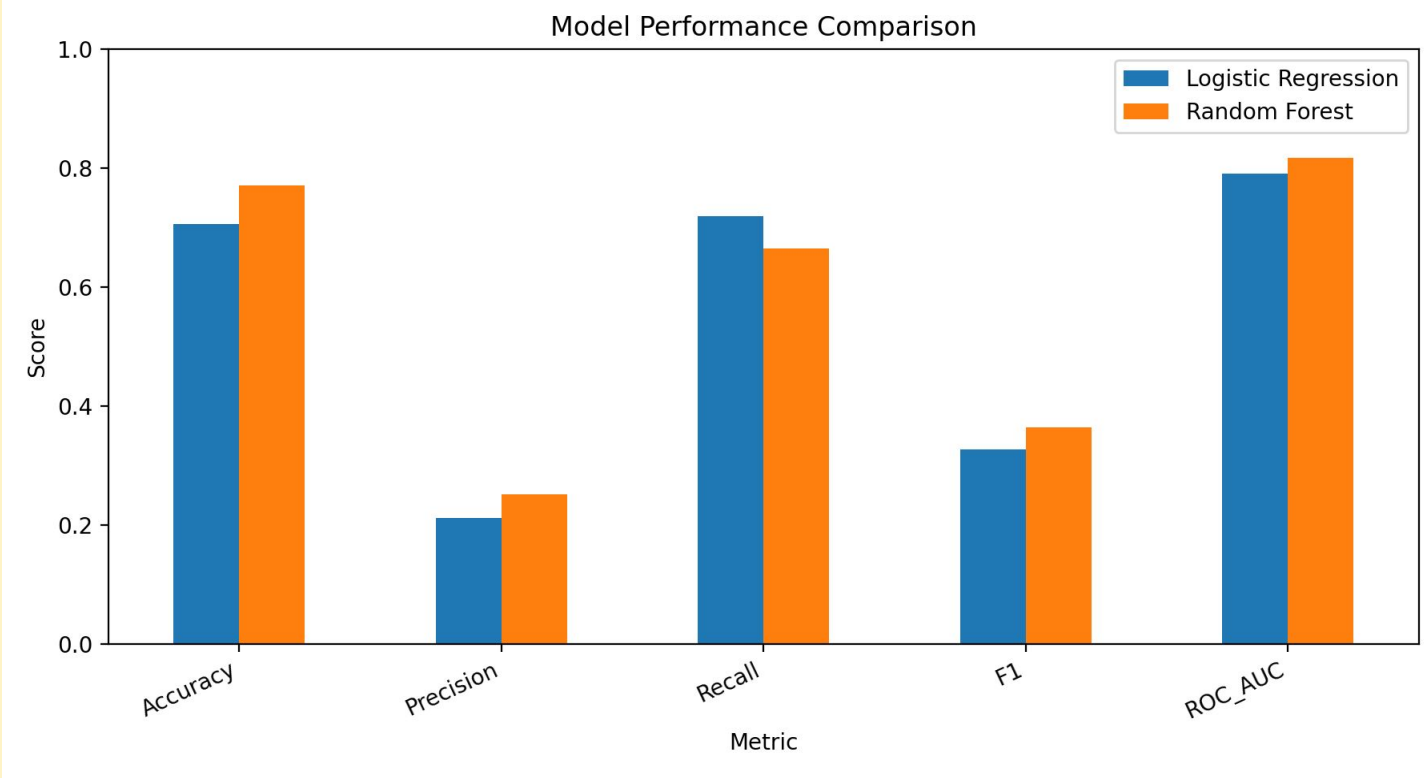
Self-Learning Tutorial

Predicting In-Hospital Mortality with MIMIC-III

Machine learning tutorial using demographics and first-24-hour lab values



Final model comparison from notebook results

Tutorial Roadmap

OVERVIEW

What a peer should be able to rebuild after following this deck

1

Load MIMIC-III tables

Admissions, patients, lab events, and lab-item dictionary.

2

Engineer early-admission features

Age, demographics, target label, and first-24-hour lab summaries.

3

Train models

Compare Logistic Regression and Random Forest using the same preprocessing pipeline.

4

Evaluate and explain

Use ROC AUC, recall, precision, F1, confusion matrices, and feature importance.

Learning goal: build a healthcare ML workflow that is explainable, reproducible, and careful about data leakage.

Analysis topic: predicting in-hospital mortality
ML methods: baseline linear model + nonlinear ensemble
Tutorial elements: commented code, plots, model interpretation, and rebuild checklist

Healthcare ML Task

PROBLEM

Predict mortality risk using information available early in the hospital stay

Prediction target

Outcome: HOSPITAL_EXPIRE_FLAG
1 = patient died during hospital admission
0 = patient survived to discharge
Unit of analysis: hospital admission, identified by HADM_ID

Why early data? It makes the prediction problem realistic and reduces leakage from future events.

```
target_col = "TARGET"

# 1 = died during admission
# 0 = survived to discharge
df["TARGET"] = df["HOSPITAL_EXPIRE_FLAG"]
```

Model inputs

Demographics and admission categories
Age calculated at admission
Lab values measured only within first 24 hours
No discharge time, death time, or length of stay used as features

Data Sources and Table Join Strategy

DATA

Four MIMIC-III tables create one modeling dataset

ADMISSIONS

Admission time, type, insurance, ethnicity, diagnosis, outcome

LABEVENTS

Time-stamped numeric lab results

PATIENTS

Gender and date of birth for age calculation

D_LABITEMS

Lab test labels used to select common labs

Final dataset: one row per hospital admission, with demographics + first-24-hour lab summaries

```
ADMISSIONS_PATH = "ADMISSIONS.csv.gz"
PATIENTS_PATH    = "PATIENTS.csv.gz"
LABEVENTS_PATH   = "LABEVENTS.csv.gz"
D_LABITEMS_PATH  = "D_LABITEMS.csv.gz"
```

Step 1: Load Data and Merge Demographics

STEP 1

Keep only columns needed for the tutorial to reduce memory usage

```
admissions = pd.read_csv(
    ADMISSIONS_PATH,
    compression="gzip",
    usecols=["SUBJECT_ID", "HADM_ID", "ADMITTIME",
             "DISCHTIME", "ADMISSION_TYPE", "INSURANCE",
             "ETHNICITY", "HOSPITAL_EXPIRE_FLAG"]
)

patients = pd.read_csv(
    PATIENTS_PATH,
    compression="gzip",
    usecols=["SUBJECT_ID", "GENDER", "DOB"]
)

df = admissions.merge(patients, on="SUBJECT_ID", how="left")
```

Tutorial design choice: load only the columns used later so a peer can run the notebook more easily.

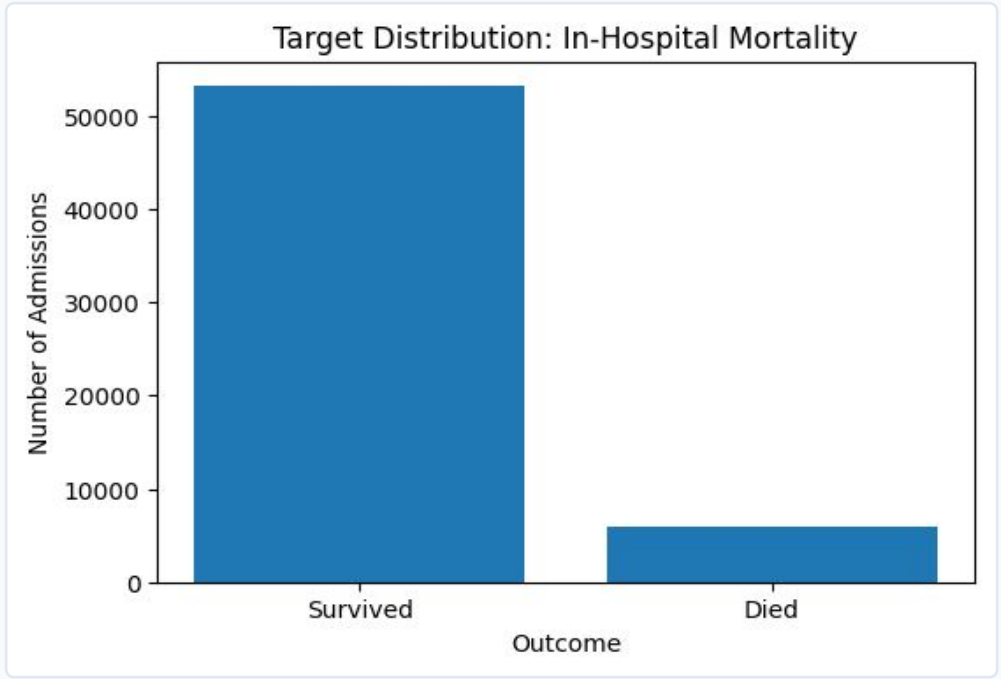
Convert ADMITTIME, DISCHTIME, and DOB to datetime
Merge admissions and patients on SUBJECT_ID
Preserve HADM_ID as the admission-level identifier
Do not load raw columns that are never used in the tutorial

Step 2: Create Target and Demographic Features

STEP 2

Outcome imbalance matters for model evaluation

```
df["AGE"] = df["ADMITTIME"].dt.year - df["DOB"].dt.year
birthday_not_reached = (
    (df["ADMITTIME"].dt.month < df["DOB"].dt.month) |
    ((df["ADMITTIME"].dt.month == df["DOB"].dt.month) &
     (df["ADMITTIME"].dt.day < df["DOB"].dt.day))
)
df.loc[birthday_not_reached, "AGE"] -= 1
df["AGE"] = df["AGE"].clip(lower=0, upper=90)
df["TARGET"] = df["HOSPITAL_EXPIRE_FLAG"]
```



Target distribution from notebook

The target is imbalanced: most admissions survive, so accuracy alone is not enough.

Use recall to measure how many deaths are detected
Use precision to understand false alarms
Use ROC AUC to compare risk-ranking ability

Step 3: Select Common Lab Tests

STEP 3

Use clinically meaningful labs that are often available early

Select a small list of common labs to keep the tutorial focused
Use D_LABITEMS to map ITEMID values to human-readable lab names
Filter LABEVENTS to selected ITEMIDs and non-missing numeric values
Convert CHARTTIME to datetime for the 24-hour window filter

```
common_labs = [
    "HEMATOCRIT", "HEMOGLOBIN", "PLATELET COUNT",
    "WHITE BLOOD CELLS", "CREATININE", "UREA NITROGEN",
    "SODIUM", "POTASSIUM", "CHLORIDE", "BICARBONATE",
    "GLUCOSE", "ANION GAP"
]

d_labitems["LABEL_UPPER"] = d_labitems["LABEL"].str.upper().str.strip()
selected_labitems = d_labitems[d_labitems["LABEL_UPPER"].isin(common_labs)]
```

Rebuild tip: start with a limited lab list first; expand only after the pipeline works end-to-end.

```
selected_itemids = selected_labitems["ITEMID"].unique().tolist()
labevents = labevents[labevents["ITEMID"].isin(selected_itemids)]
labevents = labevents.dropna(subset=["VALUENUM"])
```

Step 4: Prevent Leakage with a First-24-Hour Window

STEP 4

Use lab values collected shortly after admission

```
lab_with_admit = labevents.merge(
    admissions[["HADM_ID", "ADMITTIME"]],
    on="HADM_ID",
    how="inner"
)

lab_with_admit["HOURS_FROM_ADMIT"] = (
    lab_with_admit["CHARTTIME"] - lab_with_admit["ADMITTIME"]
).dt.total_seconds() / 3600

first_24h_labs = lab_with_admit[
    (lab_with_admit["HOURS_FROM_ADMIT"] >= 0) &
    (lab_with_admit["HOURS_FROM_ADMIT"] <= 24)
]
```

Leakage example to avoid

Using discharge time, death time, or labs after the prediction point can make the model look better than it really is.

A

Admission time

Start the clock at ADMITTIME.

B

Early lab window

Keep labs between 0 and 24 hours.

C

Prediction point

Train the model using only early information.

Step 5: Aggregate Labs into Model Features

STEP 5

Convert repeated lab measurements into one row per admission

```
lab_features = first_24h_labs.pivot_table(
    index="HADM_ID",
    columns="LABEL_UPPER",
    values="VALUENUM",
    aggfunc=["mean", "min", "max"]
)

lab_features.columns = [
    f"{stat}_{lab}".replace(" ", "_")
    for stat, lab in lab_features.columns
]
lab_features = lab_features.reset_index()

model_df = df.merge(lab_features, on="HADM_ID", how="left")
```

Notebook output: model_df shape = 58,976 rows × 54 columns

Multiple measurements of the same lab are summarized
Mean captures central tendency
Minimum and maximum capture abnormal low/high values
Missing labs are handled later through median imputation

Why aggregate? Most sklearn models expect a fixed feature matrix: one row per case and one column per feature.

Step 6: Preprocess and Split the Data

STEP 6

Use one sklearn pipeline for reproducible training and testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

Train/test split from notebook

47,180 training rows
11,796 testing rows

Stratification keeps mortality prevalence similar across train and test sets
Numeric features: median imputation + scaling
Categorical features: fill missing + one-hot encode
The same preprocessing is applied inside each model pipeline

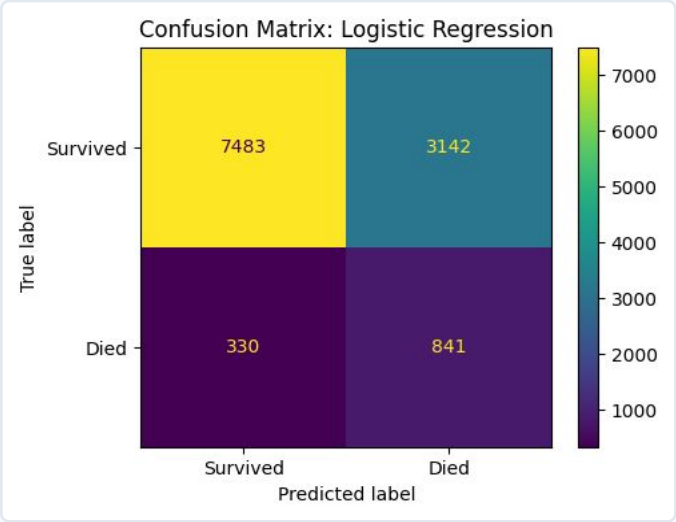
Model 1: Logistic Regression Baseline

MODEL 1

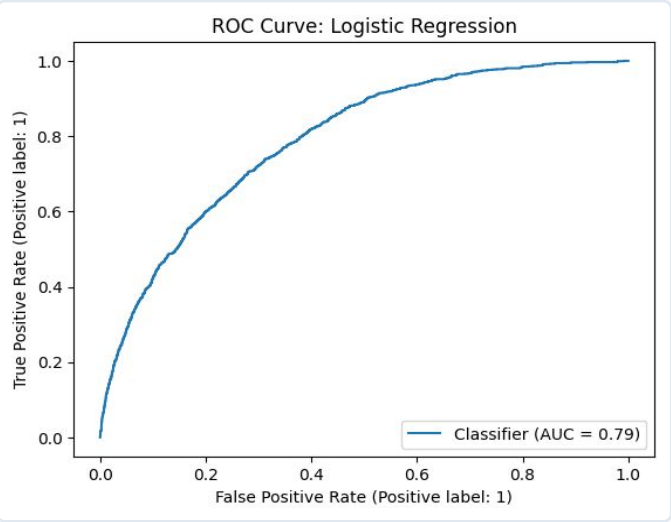
Interpretable model with balanced class weights

```
log_reg_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=42
    ))
])

log_reg_model.fit(X_train, y_train)
```



Confusion matrix



ROC curve

Result: ROC AUC = 0.791 | Recall = 0.718 | Precision = 0.211

Strength: identifies a large share of mortality cases
Weakness: many false positives because precision is low
Use case: simple baseline and interpretable comparison model

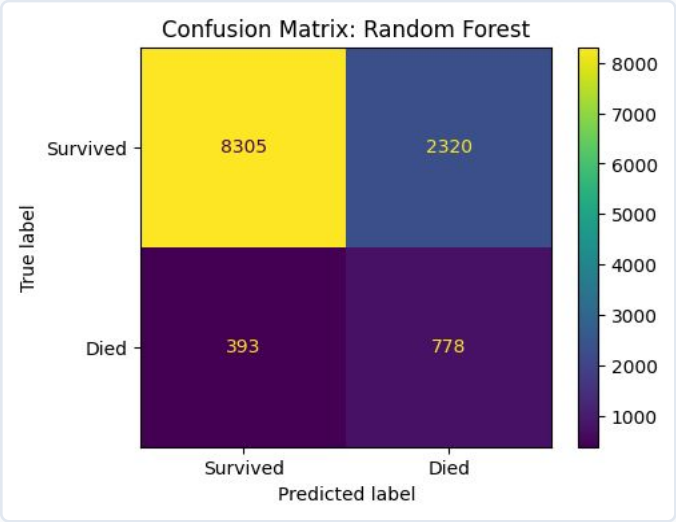
Model 2: Random Forest

MODEL 2

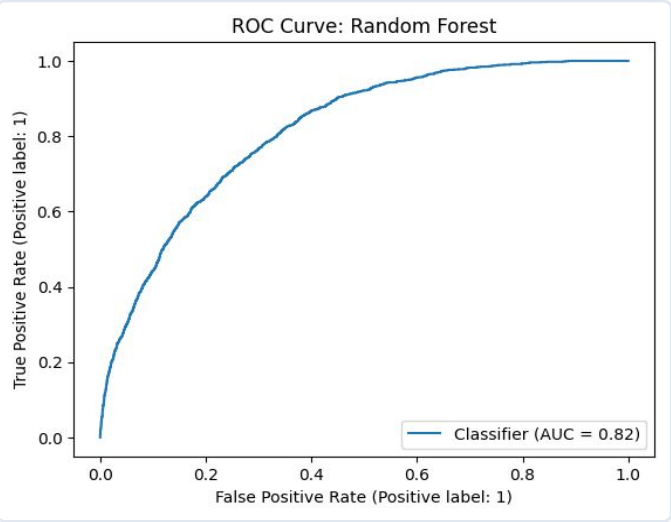
Nonlinear model that captures interactions among labs and demographics

```
rf_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=300,
        max_depth=10,
        min_samples_leaf=10,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])

rf_model.fit(X_train, y_train)
```



Confusion matrix



ROC curve

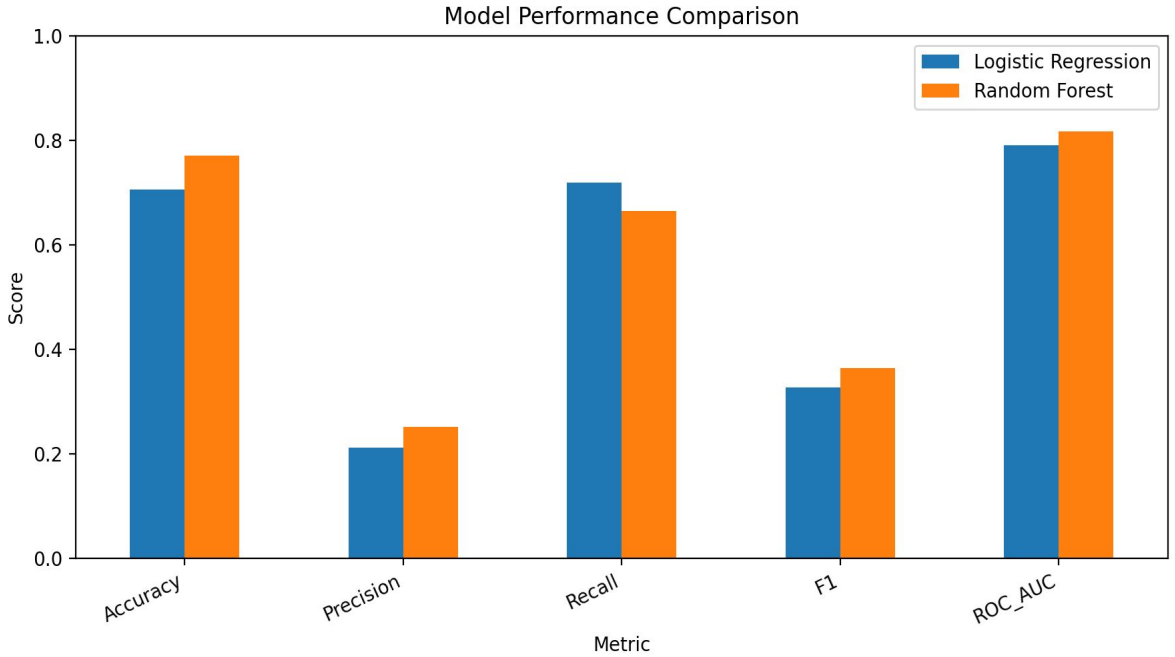
Result: ROC AUC = 0.816 | Recall = 0.664 | Precision = 0.251

Higher ROC AUC than logistic regression
Better F1 score because precision improves more than recall drops
Chosen as the final tutorial model for overall balance

Model Comparison and Final Selection

RESULTS

Random Forest performs best overall



Bar chart created from notebook metrics

Evaluation Metrics

Model	Accuracy	Precision	Recall	F1	ROC_AUC
Logistic Regression	0.706	0.211	0.718	0.326	0.791
Random Forest	0.770	0.251	0.664	0.364	0.816

Metric table

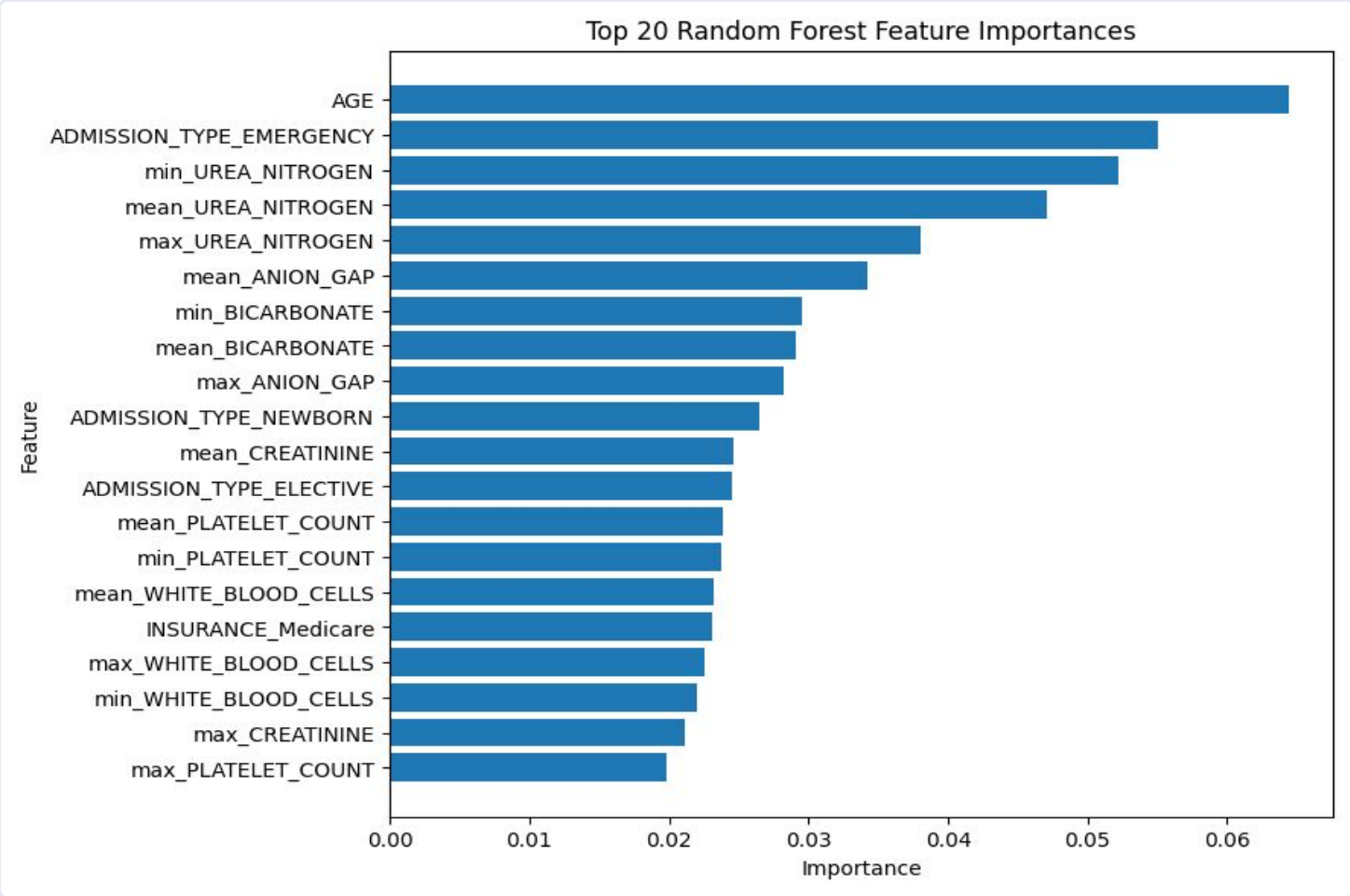
Random Forest has the highest ROC AUC: 0.816
Random Forest has the highest F1 score: 0.364
Logistic Regression has higher recall: 0.718
Clinical tradeoff: high recall reduces missed mortality cases, but low precision can cause alert fatigue

Final tutorial model: Random Forest, because it gives the best overall risk-ranking and precision/recall balance.

Explainability: Random Forest Feature Importance

EXPLAIN

Interpret which features contributed most to predictions



Top 20 Random Forest feature importances

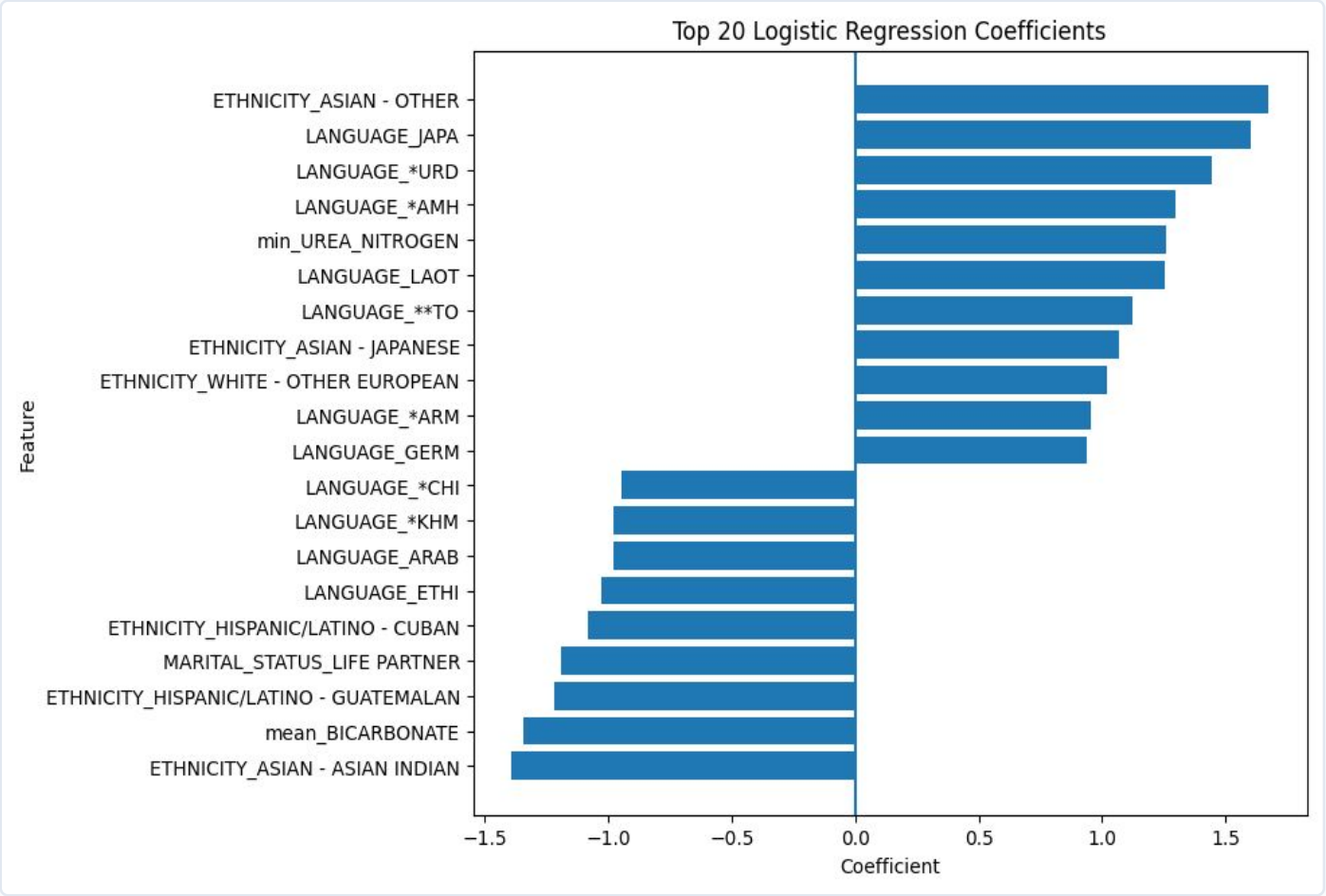
Age is the strongest predictor in this model
Emergency admission type is highly informative
Urea nitrogen, bicarbonate, anion gap, creatinine, platelet count, and white blood cell labs contribute strongly
Feature importance does not prove causality; it shows model usage

Teaching point: model explanations should be described as associations learned from data, not causal medical conclusions.

Optional Interpretation: Logistic Regression Coefficients

A second explanation view from the baseline model

EXPLAIN



Top 20 logistic regression coefficients

- Positive coefficients increase predicted mortality risk
- Negative coefficients decrease predicted mortality risk
- Because features are scaled and one-hot encoded, values are interpreted in the transformed feature space
- Use this as a teaching comparison to the random forest feature-importance plot

```
coef_df = pd.DataFrame({
    "Feature": log_reg_feature_names,
    "Coefficient": log_reg_coefficients
})
coef_df["AbsCoefficient"] = coef_df["Coefficient"].abs()
```

How to Rebuild This Tutorial

REBUILD

Checklist for peer reviewers and future reuse

Place ADMISSIONS.csv.gz, PATIENTS.csv.gz, LABEVENTS.csv.gz, and D_LABITEMS.csv.gz in the notebook directory
Run cells in order: load data → engineer features → filter first-24-hour labs → aggregate labs → train models → evaluate
Confirm no leakage variables are used: DEATHTIME, DISCHTIME, LOS_DAYS, and HOSPITAL_EXPIRE_FLAG are excluded from X
Compare models using ROC AUC, recall, precision, F1, confusion matrices, and ROC curves
Submit slides as PowerPoint to preserve speaker notes and submit code as PDF

```
# Minimal reproducibility check
print(model_df.shape)
print(results)
print(importance_df.head(20))
```

Final conclusion: first-24-hour MIMIC features can train a useful mortality-risk model, but deployment would require external validation, fairness checks, and clinical review.

Privacy reminder: do not share raw MIMIC data publicly.